

Models 1 and 2

Logical Analysis

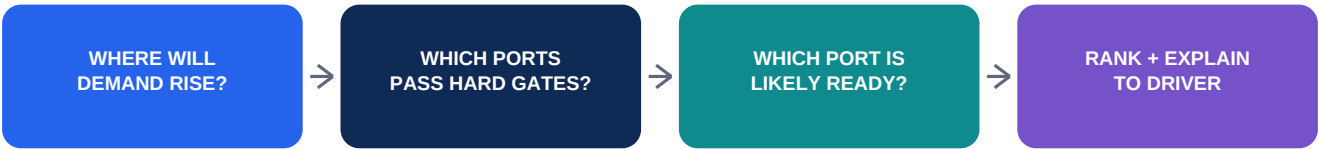
How demand forecasting and charger availability prediction work, what data they consume, how they are verified, and how FastAPI uses them safely.

MODEL 1	MODEL 2
60-minute zone demand forecast	Port availability at driver ETA
Poisson histogram boosting	Calibrated binary classifier with abstention
Synthetic-validated bundle	Backend-ready shadow bundle

01 Executive mental model

The two models answer different questions and should be used together, never interchangeably.

Question	Model 1 - Demand	Model 2 - Availability
Unit	Pune geographic zone	Individual charger port
Output	Expected request count	Probability port is unavailable
Time	60-minute window after 15-minute lead	Specific driver ETA
Decision role	Heatmaps, ranking, owner planning	Safety-aware port ranking
Algorithm	Poisson histogram gradient boosting	HGB classifier + Platt calibration
Application fallback	Seasonal historical demand	Return unknown / abstain
Current status	Synthetic-validated	Development-verified, shadow-ready

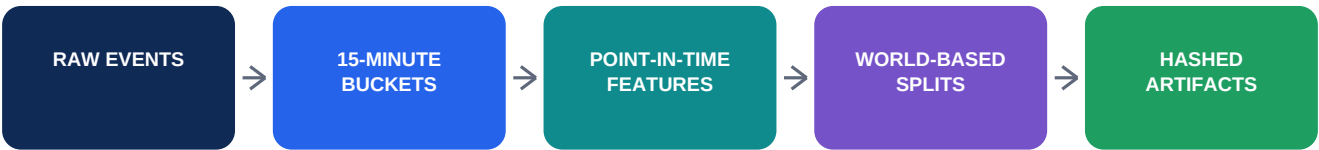


Core idea
Model 1 helps VoltEZ position the search in space and time. Model 2 evaluates uncertainty around each eligible port. PostgreSQL booking truth and compatibility rules remain authoritative throughout.

- Neither model predicts an exact human future. Both estimate useful conditional expectations or risks.
- The models never receive raw identifiers as shortcuts and never use events occurring after prediction time.
- Synthetic results validate engineering logic. Real traffic shadowing is still required before production claims.

02 Shared data foundation

Five independent 90-day Pune worlds create enough variation for training, selection, stress testing, and isolation.



Why synthetic data was necessary

VoltEZ has no live marketplace history yet, and no compatible public dataset represents Pune charger searches, unserved requests, bookings, status reports, and host-controlled availability together. The generator therefore simulates complete operational histories while preserving database relationships and timestamps.

World role	What it is allowed to do	Leakage rule
Train A + B	Fit Model 1/2 and create cross-world calibration scores	Only past events at each prediction origin
Validation	Choose architecture and Model 2 decision thresholds	Never used to fit Model 2 calibration
Stress	Challenge shift, outages, stale reports, and unusual demand	No tuning to individual stress rows
Locked test	One-time final estimate after choices are frozen	Model 2 remains unopened

Point-in-time rule

The model may know only what the app knew then

For a prediction at 10:00, a session ending at 10:05, a driver arriving at 10:20, or a status report ingested at 10:01 cannot appear in the feature vector. This is the central defense against target leakage.

- Search, request, booking, session, status, business, charger, zone, and context records remain relationally consistent.
- Unknown truth stays unknown; it is not silently relabeled as unavailable.
- Feature manifests, code commits, and SHA-256 hashes make each experiment reproducible.

03 Model 1 - what demand forecasting means

Predict the expected number of charging requests in a zone, not the exact number of people who will arrive.

Target definition

At prediction origin t , Model 1 forecasts the total request count from $t + 15$ minutes through $t + 75$ minutes. The 60-minute target is the sum of four future 15-minute buckets.

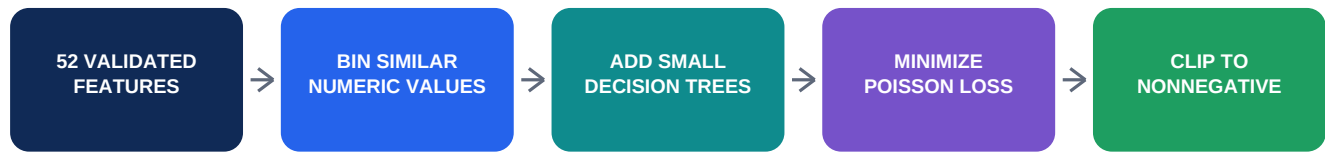
Feature family	Examples	What it tells the model
Recent demand	Prior 15/30/60-minute counts, EWM, rolling mean/std	Current momentum and volatility
Seasonality	Same time yesterday/week, target hour/day cycles	Repeated commuting and daily patterns
Supply pressure	Listed/available ports, occupancy, no-candidate rate	Whether demand is meeting constrained supply
Demand funnel	Search, request, booking, and session lags	Intent versus successfully served demand
Spatial context	Neighbor demand, zone type, centroid	Local land-use and spillover
Known context	Events, monsoon disruption, outage clusters	External changes known before prediction
Missingness	History counts and missing-lag flags	Whether absence means zero or unavailable history

Why searches alone and bookings alone are incomplete

Bookings are capacity-limited: ten people may need a charger while only two can book. Searches are noisier and may include casual browsing. Model 1 uses the funnel as context but predicts the cleaner charging-request target. That lets unserved demand influence the forecast without equating every search with a genuine charging need.

04 Model 1 - how the engine learns

Histogram gradient boosting builds a sequence of small decision trees that correct one another.



Why Poisson loss

The target is a nonnegative count with many zeros and a variance that grows with demand. Poisson loss rewards accurate conditional means without producing negative predictions. A prediction of 1.7 means the same situation would average about 1.7 requests over many comparable periods.

Mechanism	Plain-language effect	Failure if pushed too far
Learning rate	How much each new tree changes the answer	Too high learns noise; too low needs many trees
Tree leaves	How many interactions a tree can distinguish	Too many can memorize synthetic quirks
Minimum leaf rows	Evidence required before creating a rule	Too small creates fragile niche rules
L2 regularization	Penalizes extreme corrections	Too much erases useful peaks
Poisson objective	Fits expected counts and respects count variance	Still smooths rare unpredictable spikes

Example

A retail zone has rising recent requests, low available-port supply, a strong evening pattern, and a known local event. Different trees recognize each condition and their interactions; the final forecast is the sum of their corrections around the learned baseline.

05 Model 1 - verification and honest limitations

The frozen champion was selected before its one-time locked test was opened.

Metric	Validation	Stress	Locked test	Interpretation
MAE	0.742	0.756	0.739	Average absolute count error
RMSE	0.994	1.010	0.990	Penalizes larger errors
Mean bias	+0.0026	+0.0070	+0.0021	Almost no overall over/under forecast
Within one request	-	-	75.89%	Operationally close predictions
Top-decile precision	-	-	85.10%	High-ranked periods usually contain demand
Seasonal MAE improvement	-	-	22.44%	Gain over same-time historical baseline



Main limitation

The model smooths rare spikes. On locked-test periods with true demand of 3 or more, truth averaged 3.63 while predictions averaged 1.60. Use it for ranking, planning, and expected load - not as an exact capacity guarantee.

- A 10,000-row serving audit produced finite nonnegative results and exact estimator/serving agreement.
- Malformed calendars, negative counts, and missing features are rejected; major distribution shift uses seasonal fallback.
- All evidence is synthetic Pune data, so the stage remains synthetic_validated pending live shadow labels.

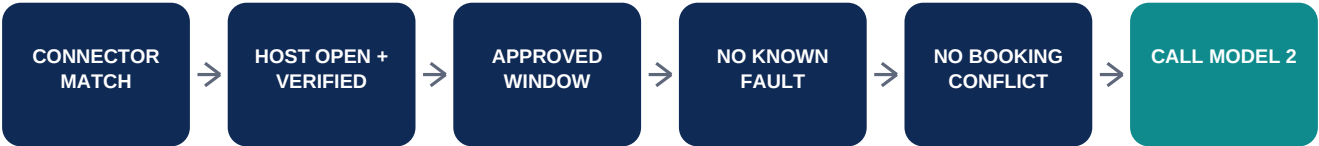
06 Model 2 - what availability prediction means

Estimate whether a physically eligible port will actually be ready around the driver's ETA.

Prediction target

P(port is unavailable within 10 minutes of ETA). Available truth requires verified service_ready_at <= ETA + 10 minutes. Unknown evidence is excluded from fitting, not forced into a false label.

Step zero: deterministic hard gates



A tree model must never overrule transaction truth. If PostgreSQL already knows a port is incompatible, closed, faulted, or booked, the backend returns unavailable without asking ML. Model 2 handles the uncertainty left after these rules.

Feature family	Examples	Logic
Status evidence	Latest status/source/confidence/age/expiry	Fresh credible reports matter more than stale ones
Driver timing	ETA, target hour/day, minutes to close	A port ready now may not be ready later
Active usage	Session count and elapsed minutes	Long-running sessions may overrun the ETA
Reliability	Prior success/failure and Bayesian smoothing	Avoid overtrusting one observation or cold-start ports
Local pressure	Nearby bookings, recent requests/unserved/occupancy	Congestion raises future readiness risk
Infrastructure	Connector, AC/DC, power, site size, access	Different port/site behavior has different risk

07 Model 2 - classifier, calibration, and abstention

A strong ranking score is not automatically an honest probability, so calibration is a separate training stage.



Balanced class weights help the classifier notice the 11-12% unavailable minority, but they make raw scores too pessimistic. The Platt sigmoid learns a mapping from out-of-world raw scores to observed risk. Validation labels are not used for calibration; they select the application thresholds.

Calibrated probability that the port will be unavailable



Band	Selection rule	Why it exists
Available	Maximum coverage while observed unavailability <= 5%	Protect driver trust
Unknown	Probability between the two evidence thresholds	Refuse false certainty
Unavailable	Maximum recall while validation precision >= 60%	Make explicit warnings reasonably trustworthy

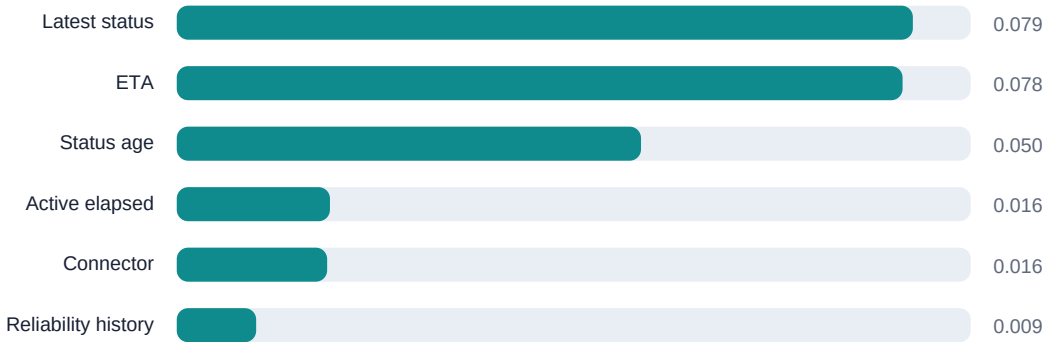
Parameter tradeoff

Raising the unavailable-precision target makes warnings more trustworthy but reduces recall. Raising the allowed available-risk target increases coverage but exposes drivers to more wrong available claims. These are product safety settings, not cosmetic tuning knobs.

08 Model 2 - verification evidence

Independent validation and stress worlds test ranking, probability quality, and the final three-state policy.

Metric	Validation (35,276)	Stress (37,452)	Meaning
ROC-AUC	0.8031	0.8088	General unavailable-vs-available ranking
PR-AUC	0.3275	0.3349	Minority-class ranking; prevalence is about 0.119
Brier score	0.0907	0.0908	Squared probability error; lower is better
Calibration error	0.0065	0.0064	Predicted risk closely tracks observed frequency
Decided accuracy	94.65%	94.67%	Accuracy after excluding unknown decisions
Unsafe available rate	4.95%	4.77%	Risk among confident available decisions
Abstention rate	30.53%	31.09%	Cases honestly returned as unknown
Unavailable precision	61.54%	53.74%	Trustworthiness of explicit warnings
Unavailable recall	4.20%	4.17%	Fraction of all unavailable cases explicitly warned



How to read the low unavailable recall

The model ranks failures reasonably well, but the current threshold makes only the safest high-risk warnings. Moderate risks become unknown and can still lower route ranking. This is a deliberate precision-first policy, not proof that failures are impossible to detect.

09 How both models work inside VoltEZ

The recommendation engine combines forecasts, hard constraints, and port-level uncertainty in a controlled order.



Stage	Input	Output	Who owns the truth
1. Candidate search	Route, vehicle connector, distance	Nearby compatible chargers	PostGIS + backend
2. Demand forecast	Zone/time historical feature snapshot	Expected requests	Model 1
3. Eligibility	Windows, bookings, faults, verification	Eligible ports only	PostgreSQL constraints
4. Availability risk	Port/status/session/reliability snapshot	Probability + three-state decision	Model 2 under contract
5. Ranking	Detour, price, demand, availability, reliability	Ordered charger alternatives	Deterministic application logic
6. Booking	Selected port and time range	Atomic reservation	Database exclusion constraint

Why the final ranker stays deterministic

ML supplies expected demand and calibrated risk. Product policy combines them with distance, price, connector compatibility, and reliability using explicit weights. This remains explainable to drivers and prevents one uncertain model from controlling bookings.

- Model 1 can identify a high-demand zone while Model 2 still rejects a risky individual port.
- A low-demand zone does not make an incompatible or booked port available.
- A Model 2 unknown is a ranking penalty or review signal, never an automatic available claim.

10 FastAPI and PostgreSQL integration

Both bundles are importable from the ML-Arn timer branch and match the backend's application-factory layout.

Component	Model 1	Model 2
Serving class	DemandPredictor	AvailabilityPredictor
Request DTO	DemandFeatureRequest	AvailabilityFeatureRequest
Response DTO	DemandPredictionResponse	AvailabilityPredictionResponse
Bundle path	models/demand/voltez-demand-60m-pune-v1	models/availability/voltez-availability-pune-v1
Internal endpoint	/api/v1/internal/ml/demand	/api/v1/internal/ml/availability
Safety response	warning or seasonal fallback	unknown / abstained

Startup pattern

Load once

FastAPI lifespan loads and hash-checks both artifacts once into app.state. Each route receives a Pydantic request, calls predict or predict_batch, and maps contract errors to HTTP 422. Loading a model inside every HTTP request wastes latency and memory.

Database and prediction audit trail

- Store the exact point-in-time payload or feature_snapshot_id in ml_lab.feature_snapshots.
- Store model_id, entity_id, target_time, output, confidence/quality, and generated_at in ml_lab.ml_predictions.
- Never overwrite historical predictions when deploying a new bundle; model versions must remain auditable.
- Use PostgreSQL exclusion constraints for overlapping bookings. Redis locks improve latency but are not the final source of truth.
- Install the ML package with Python 3.12 and the checked-in uv.lock so scikit-learn/joblib versions match training.

11 Shadow deployment, monitoring, and retraining

A correct model is a controlled lifecycle, not just a .joblib file.

Signal	Model 1	Model 2	Response
Outcome quality	MAE, WAPE, bias, within-one rate	Brier, PR-AUC, calibration, unsafe available	Investigate by zone/hour/port cohort
Coverage	Fallback rate	Unknown and hard-gate rates	Check feature outages and novel categories
Drift	Demand/zone/hour feature shift	Status age, ETA, occupancy, reliability shift	Alert; do not auto-promote retraining
System	Latency, load errors, model hash	Latency, load errors, contract rejection	Fail readiness or use safe application fallback
Fairness/product	Systematic zone underforecast	Host/access-type error slices	Review data coverage and policy impacts



Promotion criteria

- Model 1: stable real-world MAE/WAPE and bias, acceptable peak behavior, low fallback, and representative zone coverage.
- Model 2: calibrated real-world risk, unsafe-available rate within policy, useful decided coverage, and no critical cohort failures.
- Both: reproducible training commit, immutable artifact hashes, passing contracts, rollback bundle, and explicit human approval.

Do not tune on the locked test

Model 1's locked test is already consumed and must remain a historical report. Model 2's locked test may be opened once only after the final candidate and decision policy are frozen. Future tuning requires a new holdout.

12 Tooling, sponsor roles, and builder knowledge check

Sponsor integrations should perform real engineering work; they should not change model truth or appear only as logos.

Tool / sponsor	Useful role around the models	Boundary
Render	Host FastAPI, bundles, health checks, and shadow worker	No training secrets in images or logs
n8n	Schedule monitoring, label joins, reports, and human approvals	No automatic production promotion
Tavily	Discover source-attributed public events/context	Fetches text must become timestamped validated features
Google route services	Provide real ETA before Model 2 feature construction	ETA must be recorded with fetch time
Gemini / Lyrzr	Explain numeric outputs in plain language	Cannot invent, override, or recalculate predictions
CodeMate AI	Review contracts, leakage tests, and integration code	Not a source of labels or evaluation truth
Swytchcode	Govern external status/context adapter calls	Must preserve provenance and safe fallbacks

Knowledge check

- Why can Model 1 return 1.7 requests when no fractional driver exists?
- Why are hard booking conflicts checked before Model 2 rather than learned as a feature?
- Why does calibration improve probability meaning without necessarily changing ROC-AUC?
- What happens to Model 2 recall when the unavailable-precision target is raised?
- Why is unknown safer than converting missing evidence into unavailable or available?
- Which metrics would you watch first during one week of real shadow traffic, and why?

Final readiness statement

Model 1 is a synthetic-validated demand bundle with one-time locked-test evidence. Model 2 is independently validation/stress-verified and FastAPI-ready for shadow use, with its locked test still closed. Neither should be called production-proven until real VoltEZ outcomes validate the synthetic assumptions.

Evidence in repository: README.md; docs/model1_model_card.md; docs/model2_availability_training.md; docs/backend_fastapi_handoff.md; both promoted model bundles and their JSON manifests/reports.